# SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

## Smart Complaint Management System for a Community

| Project Detail | Information |
|---|---|
| Project Title | Smart Complaint Management System for a Community |
| Group Number | Team 9 |
| Course | UE24CS341A – Software Engineering |
| Development Methodology | Agile |
| Customer / Stakeholders | Resident, Administrator, Maintenance staff / Association Management (mechanic, plumber, etc.) |
| Team Lead | Riya Saju Vithayathil (PES1UG24CS378) |

## Team Members

| Team Member | SRN |
|---|---|
| Riya Saju Vithayathil | PES1UG24CS378 |
| Ritu Ranganath Kashyap | PES1UG24CS377 |
| Rohan Suresh | PES1UG24CS383 |
| Rohit Krishna R | PES1UG24CS385 |

# 1. Introduction

## Purpose

The system provides a centralised platform for residents to register and track complaints, while administrators and maintenance staff manage assignment, processing, resolution and monitoring.

## Intended Audience and Reading Suggestions

This SRS is intended for the project team, faculty/evaluators, system users, and future maintainers. Sections 2–7 describe the system, interfaces, analysis models, functional and non-functional requirements.

## Product Scope

SCMS digitises the complaint lifecycle in a residential community. Residents can submit complaints with relevant details and supporting attachments, receive a unique Complaint ID, track progress and provide feedback after resolution. Administrators manage users, categories, departments, assignments and reports. Maintenance staff process assigned complaints, update status, add remarks and record resolution.

## References

- PES University CSE Software Engineering Level 3 Student Submission Guide – Deliverable 2: Requirements.
- PES University Software Requirements Specification template, 2026.
- Synopsis / Project Proposal: Smart Complaint Management System for a community.

# 2. Overall Description

## Product Perspective

SCMS is a centralized web application. Residents, maintenance staff and administrators access the system through a web browser. The application server handles authentication, authorization, complaint workflow and business rules. MySQL provides persistent storage for users, complaints, categories, departments, assignments, status history, notifications and feedback.

## Product Functions

- User registration, authentication and profile management.
- Complaint registration with category, description, location and optional attachment.
- Automatic unique Complaint ID generation.
- Complaint categorisation and assignment to a department/staff member.
- Complaint processing, remarks and status updates.
- Resident complaint tracking and status history.
- Search and filtering.
- Resolution recording and closure.
- Feedback and rating after resolution.
- Administrative dashboard, user/category/department management and basic reports.

## Scope

| In Scope | Out of Scope |
|---|---|
| User registration and authentication | AI-based automatic complaint classification |
| Role-based access | AI-based priority prediction |
| Complaint registration | Automatic duplicate complaint detection |
| Complaint categorization | SMS/WhatsApp integration |
| Complaint assignment | Large-scale multi-organisation deployment |
| Status management and tracking | Advanced geographical complaint mapping |
| Search/filtering and attachments | Advanced predictive analytics |
| Notifications, resolution and feedback | |
| Admin dashboard and basic reports | |

## User Classes

| User Class | Privileges | Characteristics |
|---|---|---|
| Resident / Complainant | Register, submit, view and track own complaints, receive notifications, provide feedback | Non-technical; regular use |

| | | |
|---|---|---|
| Maintenance Staff / Officer | View assigned complaints, update status, add remarks, record resolution | Operational user; needs quick interaction |
| Administrator / Association Management | Manage users/staff/categories/departments, assign complaints, monitor and view reports | Higher privilege; trained/authorized user |

## Design and Implementation Constraints

- Initial implementation is a web application.
- Python Flask is the proposed backend and MySQL the proposed relational database.
- Role-based access must protect complaint and user information.
- The project will use Agile development, Git/GitHub and the course-supported engineering tools.

## Assumptions and Dependencies

- Residents have browser and network access.
- Association administrators maintain valid categories, departments and staff accounts.
- The application server can access the MySQL database.
- Users provide accurate complaint information.
- Notification delivery depends on the implemented notification mechanism.
- The initial deployment targets one residential community.

## Functional Requirements

| ID | Requirement | Priority | Verification |
|---|---|---|---|
| FR-001 | The system shall allow a resident to create an account using required details. | High | Functional test |
| FR-002 | The system shall reject registration when required information is missing or invalid. | High | Negative test |
| FR-003 | The system shall allow registered users to log in and log out. | High | Functional test |
| FR-004 | The system shall restrict functions according to the authenticated user's role. | High | Authorization test |
| FR-005 | The system shall allow an authenticated resident to submit a complaint. | High | Functional test |
| FR-006 | A complaint shall contain a title, category, description and location. | High | Positive + negative test |
| FR-007 | The system shall allow an optional supporting attachment | Medium | Functional test |

| | | | |
|---|---|---|---|
| | subject to configured restrictions. | | |
| FR-008 | The system shall assign a unique Complaint ID to every successfully submitted complaint. | High | Integration test |
| FR-009 | The system shall maintain predefined complaint categories. | Medium | Functional test |
| FR-010 | An administrator shall be able to manage complaint categories. | Medium | Functional test |
| FR-011 | An administrator shall be able to assign a complaint to an appropriate department or maintenance staff member. | High | Functional test |
| FR-012 | Maintenance staff shall be able to view complaints assigned to them. | High | Authorization + system test |
| FR-013 | Maintenance staff shall be able to update complaint status using permitted states. | High | System test |
| FR-014 | Maintenance staff shall be able to add remarks and record resolution details. | High | Functional test |
| FR-015 | Residents shall be able to view the current status of their own complaints. | High | System test |
| FR-016 | The system shall maintain relevant complaint status/action history. | High | Integration test |
| FR-017 | Authorized users shall be able to search complaints using permitted criteria. | Medium | Functional test |
| FR-018 | Authorized users shall be able to filter complaints by relevant fields such as category, status and date. | Medium | Functional test |
| FR-019 | The system shall generate notifications for important complaint lifecycle events. | Medium | System test |
| FR-020 | A resolved complaint shall be closable only through the defined workflow. | High | Negative + system test |
| FR-021 | A resident shall be able to provide feedback after the complaint is resolved. | Medium | Functional test |
| FR-022 | The system shall associate feedback and rating with the corresponding complaint. | Medium | Integration test |

| | | | |
|---|---|---|---|
| FR-023 | An administrator shall be able to manage resident/staff accounts according to permissions. | Medium | Authorization test |
| FR-024 | An administrator shall be able to monitor complaint counts and statuses through a dashboard. | Medium | System test |
| FR-025 | The system shall provide basic reports/statistics for complaint monitoring. | Medium | Functional test |

## Non-Functional Requirements

| ID | Category | Measurable Requirement | Verification |
|---|---|---|---|
| NFR-001 | Performance | 95% of normal complaint-management requests should submit within 10 seconds under the defined project test load. | Performance test |
| NFR-002 | Availability | The system shall prevent duplicate complaints when a resident retries a submission. | Operational review |
| NFR-003 | Usability | A new resident shall be able to submit a basic complaint in no more than 5 primary screens. | Usability walkthrough |
| NFR-004 | Reliability | A complaint shall not be created twice when the resident retries a submission after a temporary request failure. | Negative/integration test |
| NFR-005 | Maintainability | Business logic shall be separated into identifiable modules with automated unit tests for core services. | Code review + unit tests |
| NFR-006 | Testability | We should be able to test important parts of the system without having to run the entire final/production system. | Unit/integration test |

## Security Objectives

| ID | Security Objective | Related Requirements |
|---|---|---|
| SO-001 | Confidentiality: protect passwords and personal data from unauthorised access. | SEC-001, SEC-004 |
| SO-002 | Access control and integrity: ensure only authorised roles can create or change complaints, assignments, statuses and accounts. | SEC-002, SEC-003, SEC-005 |
| SO-003 | Accountability: keep actions traceable by recording who changed a complaint, what changed and when. | FR-016 |

## Security Requirements

| ID | Security Requirement | Validation Approach |
|---|---|---|
| SEC-001 | Passwords shall not be stored in plaintext. | Code/configuration review |

| | | |
|---|---|---|
| SEC-002 | Role-based access Residents should only access resident features, not staff or admin features. | Authorization tests |
| SEC-003 | The system should check and clean user input before using it. | Negative/security tests |
| SEC-004 | Users with invalid or expired login sessions should not be allowed to access protected pages | Security test |
| SEC-005 | Complaint attachments shall be restricted by configured file type and size. | Security/functional test |

## Business Rules / Domain Rules

- Only authenticated residents can submit complaints.
- Every successfully submitted complaint must have a unique Complaint ID.
- Residents can view and track their own complaints.
- Only authorised administrators can assign complaints or manage system-wide categories, departments and access.
- Only assigned/authorized maintenance staff can process a complaint.
- Complaint status transitions must follow the defined workflow: Submitted → Assigned → In Progress → Resolved → Closed.
- A complaint cannot be closed without resolution information.
- Feedback is available only after the complaint reaches Resolved status.

## Use Cases

| **Use Case ID** | **Use Case** | **Primary Actor** | **Precondition** | **Outcome** |
|---|---|---|---|---|
| UC-01 | Register/Login | Resident | Application available | Authenticated session |
| UC-02 | Submit Complaint | Resident | Resident authenticated | Complaint created + Complaint ID |
| UC-03 | Track Complaint | Resident | Complaint exists and belongs to resident | Current status/history displayed |
| UC-04 | Provide Feedback | Resident | Complaint resolved | Feedback recorded |
| UC-05 | Process Complaint | Maintenance Staff | Staff authenticated; complaint assigned | Status/remarks/resolution updated |
| UC-06 | Assign Complaint | Administrator | Administrator authenticated; complaint exists | Complaint assigned |
| UC-07 | Manage Users/Categories | Administrator | Administrator authenticated | Records updated |

| | | | | |
|---|---|---|---|---|
| UC-08 | View Reports | Administrator | Administrator authenticated | Report displayed |

## Validation Specification – Initial Planning

At the requirements stage, validation is planning-focused. Final test results will be produced during implementation and testing.

| Requirement Category | Planned Validation | Example |
|---|---|---|
| Functional | Black-box functional tests | Valid/invalid complaint submission |
| Security | Authentication/authorization + security testing | Resident attempts admin function |
| Performance | Load/response-time test | Concurrent complaint/status requests |
| Usability | Task-based walkthrough | New resident submits a complaint |
| Reliability | Failure/retry scenarios | Retry after temporary submission failure |
| Unit | Automated unit tests + coverage | Complaint ID/status validation |

## Project Plan

| Work Package | Tasks | Owner | Estimated Effort (person-days) | Output |
|---|---|---|---|---|
| WP1 | Requirements + use cases | All 4 | 4 | SRS |
| WP2 | Architecture + UML | Riya + Rohit | 4 | Architecture/Design |
| WP3 | User management + complaint registration | Riya | 6 | Working feature |
| WP4 | Complaint assignment + tracking | Rohit | 6 | Working feature |
| WP5 | Complaint processing + resolution | Ritu | 7 | Working feature |
| WP6 | Administration + feedback + reporting | Rohan | 7 | Working feature |
| WP7 | Testing + CI/CD + security | All 4 | 6 | Test/validation evidence |
| WP8 | Final validation + report | All 4 | 4 | Final report |

## Tool Stack

| Activity | Recommended / Proposed Tool | Evidence to Submit |
|---|---|---|
| Version control | Git + GitHub | Repository URL, commit history |
| Agile planning | Jira | Backlog, user stories, sprint board |
| CI/CD | Jenkins | Pipeline screenshot/log |
| Containerization | Docker | Dockerfile + build/run evidence |
| Testing | pytest | Test results |
| Static analysis | SonarQube or similar | Analysis report |
| API testing | Postman | Collection/results |
| Design | Draw.io / StarUML / PlantUML | UML diagrams |

# 3. External Interface Requirements

## User Interfaces

- Resident registration/login page.
- Resident dashboard showing own complaints and statuses.
- Complaint submission form.
- Complaint details/history page.
- Maintenance staff dashboard showing assigned complaints and processing actions.
- Administrator dashboard with complaint monitoring and management functions.
- Notification display.
- Feedback/rating interface.

## Software Interfaces

| Interface | Purpose |
|---|---|
| Web Browser | Provides the user interface. |
| Flask Application | Handles requests, authentication, authorization and business logic. |
| MySQL Database | Provides persistent storage. |
| Session/Authentication Layer | Maintains authenticated identity and role. |
| File Storage | Stores or references permitted complaint attachments. |

## Communications Interfaces

- Client-server communication shall use HTTP/HTTPS.
- Database communication shall use the configured MySQL connection.
- HTTPS should be used in production.

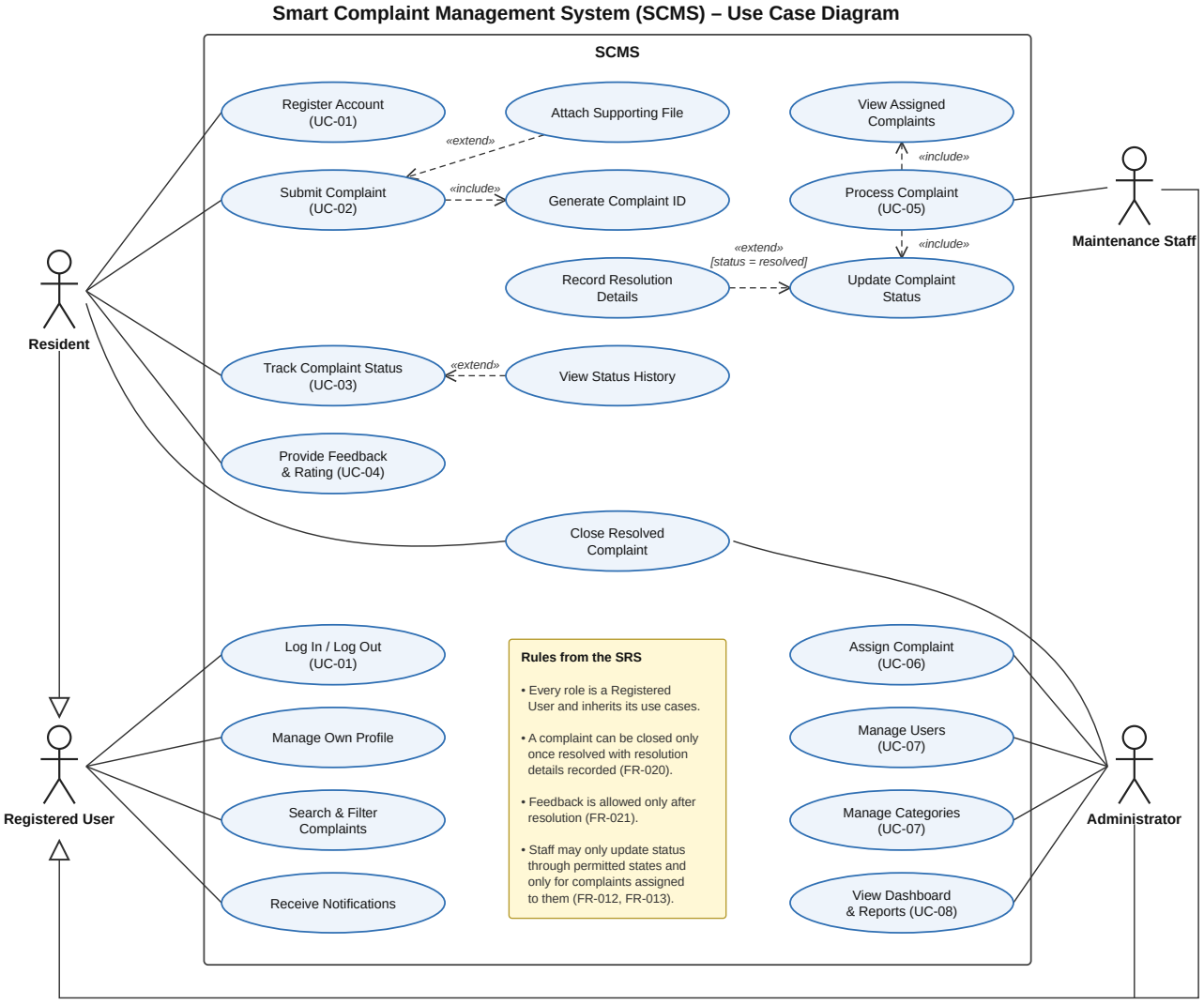
# 4. Analysis Models

The SRS analysis model identifies the actors and major system interactions. Detailed UML models are part of the subsequent design work.

## Use-Case Model

Actors: Resident, Maintenance Staff/Officer, Administrator/Association Management.

Major interactions: Register/Login, Submit Complaint, Track Complaint, Provide Feedback, Process Complaint, Assign Complaint, Manage Users/Categories, and View Reports.



## Domain/ER Model

The conceptual data model is expected to include entities such as User, Complaint, Category, Department, Assignment, Status History, Notification and Feedback. The detailed ER/domain model will be finalized during design.

## Analysis-to-Requirement Mapping

| Analysis Element | Related Requirements |
|---|---|
| Resident use cases | FR-001–FR-008, FR-015–FR-022 |
| Staff use cases | FR-012–FR-014, FR-020 |
| Administrator use cases | FR-009–FR-011, FR-023–FR-025 |
| Security model | SEC-001–SEC-005 |
| Quality model | NFR-001–NFR-006 |

# 5. System Features

| System Feature | Stimulus / Input | Response / Expected Behavior | Related FRs |
|---|---|---|---|
| 5.1 User Management & Authentication | Resident selects Register/Login and provides credentials. | System validates details, creates/authenticates the account and establishes the appropriate role/session. | FR-001–FR-004 |
| 5.2 Complaint Registration | Authenticated resident selects Submit Complaint and enters required information. | System validates the form, stores the complaint and generates a unique Complaint ID. | FR-005–FR-008 |
| 5.3 Categorization & Assignment | Administrator selects a complaint and assigns category/department/staff . | System stores the category and assignment and exposes the permitted assignment information. | FR-009–FR-011 |

| | | | |
|---|---|---|---|
| 5.4 Complaint Processing & Status | Assigned staff opens a complaint and updates status, remarks or resolution. | System validates the action, updates the complaint and records relevant history. | FR-012–FR-016 |
| 5.5 Search & Tracking | User enters a search/filter criterion or opens a complaint. | System displays only complaints permitted for that user's role, including current status/history. | FR-015–FR-018 |
| 5.6 Notifications | A significant complaint event occurs. | System generates the appropriate notification for affected users. | FR-019 |
| 5.7 Resolution & Closure | Staff records resolution and completes the defined workflow. | System stores resolution and permits closure only when business rules are satisfied. | FR-020 |
| 5.8 Feedback & Rating | Resident opens a resolved complaint and submits feedback/rating. | System validates eligibility and stores the feedback against the complaint. | FR-021–FR-022 |
| 5.9 Administration & Reporting | Administrator opens management/reporting functions. | System displays authorized management controls, complaint summaries and basic reports. | FR-023–FR-025 |

# 6. Other Nonfunctional Requirements

## Performance Requirements

- 95% of normal complaint-management requests should submit within 10 seconds under the defined test load.
- Search/filter operations should return results without unnecessary delay.

## Safety Requirements

- Critical complaint records shall be protected from accidental deletion through authorization and confirmation.
- Complaint history required for accountability should be preserved.

## Security Requirements

- Security requirements are defined in Section 2.9 and will be validated through authorization, negative and security testing.

## Software Quality Attributes

- Usability: simple resident complaint submission and tracking.
- Reliability: complaint records and states remain consistent.
- Maintainability: modular, documented and version-controlled code.
- Scalability: design should support growth in users and complaints.
- Testability: core services expose testable interfaces.

### Business Rules

- Complaint IDs are unique.
- Only authorized users can perform role-specific actions.
- Status transitions follow the defined complaint lifecycle.
- A complaint requires resolution information before closure.
- Feedback is available after resolution.

## 7. Other Requirements

- Persistent complaint and user data shall be stored in a relational database.
- Important lifecycle events should have timestamps.
- Backup and recovery procedures should be provided for deployment.
- The system should support Docker-based deployment.
- Source code and documentation shall be maintained in Git/GitHub.
- Data retention and any community-specific legal/privacy requirements shall be reviewed before deployment.

## Appendix A: Glossary

| Term | Meaning |
|---|---|
| SCMS | Smart Complaint Management System |
| SRS | Software Requirements Specification |
| Resident / Complainant | Person who submits and tracks a complaint |
| Maintenance Staff / Officer | Authorized person who processes assigned complaints |
| Administrator | Authorized person who manages the system |
| Complaint ID | Unique identifier assigned to a complaint |
| RBAC | Role-Based Access Control |
| CRUD | Create, Read, Update, Delete |

## Appendix B: Field Layouts / Report Requirements

## Resident Fields

| Field | Length / Format | Data Type | Description | Mandatory |
|---|---|---|---|---|
| resident_id | 36 | UUID/String | Unique resident identifier | Y |
| name | 100 | String | Resident name | Y |

| email | 100 | String | Unique login email | Y |
|---|---|---|---|---|
| password | — | Hashed String | Authentication credential | Y |
| contact_number | 15 | String | Contact number | Y |
| unit_address | 150 | String | Residential unit/address | Y |
| role | 20 | Enum/String | User role | Y |

## Complaint Fields

| Field | Length / Format | Data Type | Description | Mandatory |
|---|---|---|---|---|
| complaint_id | 36 | UUID/String | Unique complaint identifier | Y |
| resident_id | 36 | UUID/String | Resident who submitted complaint | Y |
| title | 150 | String | Short complaint title | Y |
| category_id | 36 | UUID/String | Complaint category | Y |
| description | 1000 | Text | Detailed complaint description | Y |
| location | 200 | String | Issue location | Y |
| attachment | Configured | File/Path | Supporting evidence | N |
| status | 20 | Enum/String | Complaint state | Y |
| created_at | DateTime | DateTime | Submission timestamp | Y |
| updated_at | DateTime | DateTime | Last update timestamp | Y |

## Report Requirements

- Total complaints.
- Complaints by status: Submitted, Assigned, In Progress, Resolved and Closed.
- Complaints by category.
- Complaints by date/period.
- Pending and unresolved complaints.
- Basic staff/department workload information.

# Appendix C: Requirement Traceability Matrix

| Requirement | Architecture / Module | Design | Code | Test |
|---|---|---|---|---|
| FR-001–FR-004 | Authentication/User Module | Authentication design | To be implemented | Functional + authorization tests |
| FR-005–FR-008 | Complaint Module | Complaint registration design | To be implemented | Functional + integration tests |

| | | | | |
|---|---|---|---|---|
| FR-009–FR-011 | Category/Assignment Module | Assignment workflow | To be implemented | Functional + integration tests |
| FR-012–FR-016 | Processing/Tracking Module | Status/history design | To be implemented | System + integration tests |
| FR-017–FR-020 | Search/Notification/Closure | Workflow design | To be implemented | Functional/system tests |
| FR-021–FR-025 | Feedback/Admin/Reports | Admin/report design | To be implemented | System/acceptance tests |
| NFR-001–NFR-006 | Application/Infrastructure | Quality design | To be implemented | Performance/usability/unit/reliability tests |
| SEC-001–SEC-005 | Authentication/Authorization | Security design | To be implemented | Security/authorization tests |